

This project is about taking the shortest path to arrive to your location so we used 4 algorithms to see which one is the fastest and gives a good result.

The algorithms that we used are:

- Dijkstra's Algorithm
- Bellman-Ford Algorithm
- Floyd-Warshall Algorithm
- A* (A-Star) Algorithm

Dijkstra's Algorithm

Dijkstra's algorithm is one of the algorithms most people use daily; it's mostly used in GPS apps.

How does it work?

First, it sets the distance to the start node to **0** and all other nodes to infinity after that, it will pick the unvisited node with the smallest distance, then it looks at all neighbors of that node and Calculate the distance to those neighbors through the current node and update the path that it will take (If this new path is shorter than the previous path, then it will update the path that it will take).

Bellman-Ford Algorithm

Bellman-Ford Algorithm it's a algorithm that can deal with negative weights its more powerful than Dijkstra's algorithm, but it's not as functional in Ur project because it takes a long time to give the output, and the negative weight is rare in node maps. This algorithm is also used in GPS apps.

How does it work?

As Dijkstra's algorithm it look at all the possible ways to go to its location, adding to that, Bellman-Ford Algorithm looks at another thing, which is a path that contains a negative value. By negative, I mean it can make the time of arrival shorter, like using shortcuts.

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm is the slowest one and will not be useful for our project. However it's a reliable way to look for the shortest path and is mostly used in Networking and Telecommunications.

How does it work?

Floyd-Warshall Algorithm it visit all the nodes and comparing which path is the best path.

The thing that makes it slow is that it visits all the nodes.

A* (A-Star) Algorithm

A* algorithm is the fastest algorithm, but it's risky. This algorithm is used at pathfinding and graph traversal where it need to find the result fast.

How does it work?

First, it looks at the destination node thin it go to it without any seeing any other path. The downside of it is that it may go to a dead end without knowing that the path it taking is not correct

Samples

We took each algorithm and tested it 3 times so we can know which one is the safest one for our project, and the test results are:

A* samples samples:

```
Cities available (500 total):
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111
Shortest path from City1 to City2: City1 -> City2
Total distance: 48.00
Computation time: 0.001068 seconds

Cities available (500 total):
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111
Shortest path from City1 to City250: City1 -> City4 -> City5 -> City9 -> City14 -> City17 -> City19 -> City24 -> City27 -> City3
Total distance: 1009.00
Computation time: 0.004479 seconds

Cities available (500 total):
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111
Shortest path from City1 to City500: City1 -> City4 -> City5 -> City9 -> City14 -> City17 -> City19 -> City24 -> City27 -> City3
Total distance: 2005.00
Computation time: 0.007647 seconds
```

Floyd-Warshall Algorithm samples

```
Cities available (500 total):
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111
Shortest path from City1 to City250: City1 -> City4 -> City5 -> City9 -> City14 -> City17 -> City19 -> City24 -> City27 -> City3
Total distance: 1009.00
Computation time: 37.480848 seconds

Cities available (500 total):
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111
Shortest path from City1 to City2: City1 -> City2
Total distance: 48.00
Computation time: 36.455700 seconds

Cities available (500 total):
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111
Shortest path from City1 to City500: City1 -> City4 -> City5 -> City9 -> City14 -> City17 -> City19 -> City24 -> City27 -> City32
Total distance: 1981.00
Computation time: 36.545628 seconds
```

Dijkstra's Algorithm samples

```
Cities available (500 total):
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111
Shortest path from City1 to City250: City1 -> City4 -> City5 -> City9 -> City14 -> City17 -> City19 -> City24 -> City27 -> City3
Computation time: 0.004336 seconds

Cities available (500 total):
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111
Shortest path from City1 to City500: City1 -> City4 -> City5 -> City9 -> City14 -> City17 -> City19 -> City24 -> City27 -> City3
Computation time: 0.004122 seconds

Cities available (500 total):
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111
Shortest path from City1 to City2: City1 -> City2
Computation time: 0.003789 seconds
```

Bellman-Ford Algorithm samples

```
Cities available (500 total):  
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111  
Shortest path from City1 to City500: City1 -> City4 -> City5 -> City9 -> City14 -> City17 -> City19 -> City24 -> City27 -> City3  
Computation time: 1.907371 seconds
```

```
Cities available (500 total):  
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111  
Shortest path from City1 to City2: City1 -> City2  
Computation time: 1.941671 seconds
```

```
Cities available (500 total):  
City1, City10, City100, City101, City102, City103, City104, City105, City106, City107, City108, City109, City11, City110, City111  
Shortest path from City1 to City250: City1 -> City4 -> City5 -> City9 -> City14 -> City17 -> City19 -> City24 -> City27 -> City3  
Computation time: 1.878908 seconds
```

Analysis

So, according to those samples and the algorithm characteristics, Dijkstra's Algorithm is the best algorithm for our project. A* algorithm and Bellman-Ford Algorithm are average, lastly, Floyd-Warshall Algorithm is the worst algorithm.

summary

Algorithm	Primary Use Case	Key Strength	Main Weakness	Project Ranking
Dijkstra's	GPS / Navigation	Always finds the shortest path efficiently.	Cannot handle negative weights.	Best Algorithm
A (A-Star)*	Games / Fast Pathfinding	Extremely fast; uses a "guess" to find the goal.	Can get stuck in dead ends if the guess is wrong.	Average Algorithm
Bellman-Ford	Financial Networks	Can handle negative weights (shortcuts).	Much slower than Dijkstra.	Average Algorithm

Floyd-Warshall	Network Infrastructure	Finds paths between all nodes at once.	The slowest	Worst Algorithm
-----------------------	-------------------------------	---	--------------------	------------------------